

C# - Phần 9.10

Đáp án code, đáp án lý thuyết và tổng kết

Student Course Registration Manager

Phạm vi: Tài liệu này tổng hợp từ bài 9.10 đã làm: bản code đã được chỉnh sạch theo các lỗi đã nhận xét, 18 câu đáp án lý thuyết và phần hệ thống hóa Delegate - Lambda - Event - Publisher/Subscriber.

PHẦN A - ĐÁP ÁN CODE ĐÃ CHỈNH

1. Student.cs

```
public class Student
{
    public string Id { get; }
    public string FullName { get; }

    public Student(string id, string fullName)
    {
        if (string.IsNullOrEmpty(id))
        {
            throw new ArgumentException(
                "Id cannot be null or whitespace.",
                nameof(id));
        }

        if (string.IsNullOrEmpty(fullName))
        {
            throw new ArgumentException(
                "Name cannot be null or whitespace.",
                nameof(fullName));
        }

        Id = id.Trim();
        FullName = fullName.Trim();
    }

    public override string ToString()
    {
        return $"Id: {Id}, Full name: {FullName}";
    }
}
```

2. Course.cs

```
public class Course
{
    public string Id { get; }
    public string Name { get; }
    public int Credits { get; }
    public decimal FeePerCredit { get; }

    public Course(
        string id,
        string name,
        int credits,
        decimal feePerCredit)
    {
        if (string.IsNullOrEmpty(id))
        {
            throw new ArgumentException(
```

```

        "Id cannot be null or whitespace.",
        nameof(id));
    }

    if (string.IsNullOrEmpty(name))
    {
        throw new ArgumentException(
            "Name cannot be null or whitespace.",
            nameof(name));
    }

    if (credits <= 0)
    {
        throw new ArgumentOutOfRangeException(
            nameof(credits),
            "Credits must be greater than 0.");
    }

    if (feePerCredit <= 0)
    {
        throw new ArgumentOutOfRangeException(
            nameof(feePerCredit),
            "Fee per credit must be greater than 0.");
    }

    Id = id.Trim();
    Name = name.Trim();
    Credits = credits;
    FeePerCredit = feePerCredit;
}

public override string ToString()
{
    return $"Id: {Id}, Name: {Name}, " +
        $"Credits: {Credits}, " +
        $"Fee Per Credit: {FeePerCredit}";
}
}

```

3. Registration.cs

```

public class Registration
{
    public Student Student { get; }
    public Course Course { get; }
    public DateTime RegisteredAt { get; }

    public Registration(Student student, Course course)
    {
        if (student == null)
        {
            throw new ArgumentNullException(
                nameof(student),
                "Student cannot be null.");
        }

        if (course == null)
        {
            throw new ArgumentNullException(
                nameof(course),
                "Course cannot be null.");
        }

        Student = student;
        Course = course;
        RegisteredAt = DateTime.Now;
    }
}

```

```
}  
}
```

4. RegistrationMessageDelegate.cs

```
public delegate void RegistrationMessageDelegate(string message);
```

5. RegistrationMessageService.cs

```
public static class RegistrationMessageService  
{  
    public static void ShowSuccessMessage(string message)  
    {  
        Console.WriteLine($"SUCCESS: {message}");  
    }  
  
    public static void WriteConsoleMessage(string message)  
    {  
        Console.WriteLine($"MESSAGE: {message}");  
    }  
}
```

6. EmailService.cs

```
public class EmailService  
{  
    public void SendRegistrationEmail(Registration registration)  
    {  
        Console.WriteLine(  
            $"Send Registration Email: " +  
            $"{registration.Student}, {registration.Course}");  
    }  
}
```

7. LogService.cs

```
public class LogService  
{  
    public void WriteRegistrationLog(Registration registration)  
    {  
        Console.WriteLine(  
            $"Write Registration Log: " +  
            $"{registration.Student}, {registration.Course}");  
    }  
}
```

8. NotificationService.cs

```
public class NotificationService  
{  
    public void ShowRegistrationNotification(  
        Registration registration)  
    {  
        Console.WriteLine(  
            $"Show Registration Notification: " +  
            $"{registration.Student}, {registration.Course}");  
    }  
}
```

9. RegistrationService.cs

Điểm chính: Đây là Publisher. Method RegisterCourse xử lý validation, tìm Student/Course, kiểm tra đăng ký trùng, tạo và lưu Registration, sau đó mới phát Event.

```
public class RegistrationService  
{  
    private readonly List<Student> students;  
    private readonly List<Course> courses;
```

```

private readonly List<Registration> registrations;

public event Action<Registration>? CourseRegistered;

public RegistrationService()
{
    students = new List<Student>();
    courses = new List<Course>();
    registrations = new List<Registration>();
}

public void AddStudent(Student student)
{
    if (student == null)
    {
        throw new ArgumentNullException(
            nameof(student),
            "Student cannot be null.");
    }

    if (students.Exists(
        foundStudent => foundStudent.Id == student.Id))
    {
        throw new InvalidOperationException(
            $"Student with id {student.Id} already exists.");
    }

    students.Add(student);
}

public void AddCourse(Course course)
{
    if (course == null)
    {
        throw new ArgumentNullException(
            nameof(course),
            "Course cannot be null.");
    }

    if (courses.Exists(
        foundCourse => foundCourse.Id == course.Id))
    {
        throw new InvalidOperationException(
            $"Course with id {course.Id} already exists.");
    }

    courses.Add(course);
}

public void RegisterCourse(
    string studentId,
    string courseId)
{
    if (string.IsNullOrWhiteSpace(studentId))
    {
        throw new ArgumentException(
            "Student Id cannot be null or whitespace.",
            nameof(studentId));
    }

    if (string.IsNullOrWhiteSpace(courseId))
    {
        throw new ArgumentException(
            "Course Id cannot be null or whitespace.",
            nameof(courseId));
    }
}

```

```

        string normalizedStudentId = studentId.Trim();
        string normalizedCourseId = courseId.Trim();

        Predicate<Student> foundStudentPredicate =
            student =>
                student.Id == normalizedStudentId;

        Student? studentRegistered =
            students.Find(foundStudentPredicate);

        Course? courseRegistered =
            courses.Find(
                course => course.Id == normalizedCourseId);

        if (studentRegistered == null)
        {
            throw new InvalidOperationException(
                $"Student with id {normalizedStudentId} " +
                "does not exist.");
        }

        if (courseRegistered == null)
        {
            throw new InvalidOperationException(
                $"Course with id {normalizedCourseId} " +
                "does not exist.");
        }

        bool alreadyRegistered =
            registrations.Exists(
                registration =>
                    registration.Student.Id ==
                        normalizedStudentId
                    &&
                    registration.Course.Id ==
                        normalizedCourseId);

        if (alreadyRegistered)
        {
            throw new InvalidOperationException(
                $"Student {normalizedStudentId} has already " +
                $"registered for course {normalizedCourseId}.");
        }

        Registration registration =
            new Registration(
                studentRegistered,
                courseRegistered);

        registrations.Add(registration);

        CourseRegistered?.Invoke(registration);
    }
}

```

10. Program.cs - dữ liệu và kịch bản test

```

public class Program
{
    public static void Main(string[] args)
    {
        Student student1 =
            new Student("SV01", "Nguyen Van A");

        Student student2 =
            new Student("SV02", "Tran Thi B");
    }
}

```

```

Course course1 =
    new Course(
        "C01",
        "C# Programming",
        3,
        500000);

Course course2 =
    new Course(
        "C02",
        "Database",
        4,
        450000);

Course course3 =
    new Course(
        "C03",
        "Web Development",
        3,
        600000);

EmailService emailService = new EmailService();
LogService logService = new LogService();
NotificationService notificationService =
    new NotificationService();

RegistrationService registrationService =
    new RegistrationService();

// Event: subscribe Email + Log
registrationService.CourseRegistered +=
    emailService.SendRegistrationEmail;

registrationService.CourseRegistered +=
    logService.WriteRegistrationLog;

// Add data
registrationService.AddStudent(student1);
registrationService.AddStudent(student2);

registrationService.AddCourse(course1);
registrationService.AddCourse(course2);
registrationService.AddCourse(course3);

// Registration 1
registrationService.RegisterCourse("SV01", "C01");

Console.WriteLine("-----");

// Add Notification subscriber
registrationService.CourseRegistered +=
    notificationService.ShowRegistrationNotification;

// Registration 2
registrationService.RegisterCourse("SV02", "C02");

Console.WriteLine("-----");

// Unsubscribe Email
registrationService.CourseRegistered -=
    emailService.SendRegistrationEmail;

// Registration 3
registrationService.RegisterCourse("SV01", "C03");

Console.WriteLine("-----");

```

```

// Duplicate registration
try
{
    registrationService.RegisterCourse("SV01", "C01");
}
catch (InvalidOperationException ex)
{
    Console.WriteLine(ex.Message);
}

// Student not found
try
{
    registrationService.RegisterCourse("SV99", "C01");
}
catch (InvalidOperationException ex)
{
    Console.WriteLine(ex.Message);
}

// Course not found
try
{
    registrationService.RegisterCourse("SV01", "C99");
}
catch (InvalidOperationException ex)
{
    Console.WriteLine(ex.Message);
}

// Invalid input
try
{
    registrationService.RegisterCourse(" ", "C01");
}
catch (ArgumentException ex)
{
    Console.WriteLine(ex.Message);
}

// Func<Course, decimal>
Func<Course, decimal> calculateFee =
    course =>
        course.Credits * course.FeePerCredit;

Console.WriteLine(
    $"Course fee: {calculateFee(course1)}");

// Action<Registration> + Statement Lambda
Registration sampleRegistration =
    new Registration(student1, course1);

Action<Registration> displayRegistration =
    registration =>
    {
        Console.WriteLine(
            $"Student: " +
            $"{registration.Student.FullName}");

        Console.WriteLine(
            $"Course: " +
            $"{registration.Course.Name}");

        Console.WriteLine(
            $"Credits: " +
            $"{registration.Course.Credits}");
    }

```

```

        Console.WriteLine(
            $"Registered at: " +
            $"{registration.RegisteredAt}");
    };

    displayRegistration(sampleRegistration);

    Console.WriteLine("-----");

    // Custom Delegate
    RegistrationMessageDelegate messageDelegate =
        RegistrationMessageService.ShowSuccessMessage;

    messageDelegate("Completed");

    Console.WriteLine("-----");

    // Multicast Delegate
    messageDelegate +=
        RegistrationMessageService.WriteConsoleMessage;

    messageDelegate("Completed");
}
}

```

PHẦN B - ĐÁP ÁN 18 CÂU LÝ THUYẾT

1. Delegate trong C# là gì?

Delegate là một kiểu trong C# có thể tham chiếu đến một hoặc nhiều method có chữ ký tương thích với Delegate đó.

2. Multicast Delegate là gì?

Multicast Delegate là Delegate có thể chứa nhiều method trong invocation list. Khi Delegate được gọi, các method đã đăng ký sẽ lần lượt được thực thi.

3. Action khác Func ở điểm nào?

Action dùng cho method không trả về giá trị (void). Func dùng cho method có giá trị trả về; generic cuối cùng của Func là kiểu return.

4. Predicate<T> có chữ ký như thế nào?

Predicate<T> nhận một giá trị kiểu T và trả về bool, tức T -> bool. T có thể là reference type hoặc value type.

5. Với Func<Course, decimal>, Course và decimal có vai trò gì?

Course là kiểu tham số đầu vào; decimal là kiểu giá trị trả về.

6. Lambda student => student.Id == normalizedStudentId nhận và trả kiểu gì?

student có kiểu Student. Biểu thức so sánh trả về bool, nên toàn bộ Lambda có dạng Student -> bool.

7. Vì sao Lambda đó truyền được vào Find() và Exists()?

Vì Lambda có chữ ký Student -> bool, tương thích với Predicate<Student>. Find() và Exists() đều nhận Predicate<Student>. Find() trả Student phù hợp hoặc null, còn Exists() trả bool.

8. Event trong C# được xây dựng dựa trên kiến thức nào?

Event được xây dựng dựa trên Delegate. Từ khóa event bổ sung khả năng kiểm soát: code bên ngoài có thể subscribe/unsubscribe nhưng không được tùy ý gán hoặc raise Event.

9. += và -= với Event dùng làm gì?

+= dùng để đăng ký Event Handler; -= dùng để hủy đăng ký Event Handler.

10. CourseRegistered?.Invoke(registration) có nghĩa gì?

Nếu Event không null thì Invoke sẽ gọi tất cả Event Handler đang đăng ký và truyền registration cho các handler. Nếu Event null thì bỏ qua.

11. Vì sao RegistrationService là Publisher?

Vì RegistrationService sở hữu Event CourseRegistered, quyết định khi nào Event xảy ra và là nơi phát Event bằng Invoke.

12. Subscriber trong project là những class nào?

EmailService, LogService và NotificationService. RegistrationMessageService không phải Subscriber của CourseRegistered; nó chỉ phục vụ Custom/Multicast Delegate.

13. SendRegistrationEmail(), WriteRegistrationLog(), ShowRegistrationNotification() là gì?

Sau khi được đăng ký bằng +=, chúng là các Event Handler.

14. Vì sao RegistrationService không cần biết EmailService tồn tại?

Vì RegistrationService chỉ phát CourseRegistered. Việc Subscriber nào lắng nghe Event được cấu hình từ bên ngoài, ví dụ trong Program. Nhờ đó Publisher giảm phụ thuộc trực tiếp vào Subscriber.

15. Nếu thêm SmsService, có cần sửa RegistrationService không?

Không. Chỉ cần tạo SmsService có handler phù hợp rồi subscribe handler đó vào CourseRegistered bằng +=.

16. Event và Publisher/Subscriber có cùng khái niệm không?

Không. Event là cơ chế/công cụ của C# dựa trên Delegate. Publisher/Subscriber là cách tổ chức các object sử dụng Event: Publisher sở hữu và phát Event; Subscriber đăng ký handler để nhận thông báo.

17. Vì sao RegisterCourse() phải kiểm tra đăng ký trùng?

Để một Student không đăng ký cùng một Course nhiều lần, tránh dữ liệu trùng và giữ tính nhất quán của registrations.

18. Vì sao phải Add Registration trước rồi mới Invoke Event?

Vì chỉ nên thông báo đăng ký thành công sau khi dữ liệu đã được lưu thành công. Nếu phát Event trước, Subscriber có thể nhận thông báo thành công trong khi việc lưu dữ liệu thực tế thất bại.

PHẦN C - TỔNG KẾT LÝ THUYẾT PHẦN 9

1. Delegate và Multicast Delegate

Delegate là kiểu tham chiếu đến method có chữ ký tương thích.

```
public delegate void MessageDelegate(string message);

MessageDelegate handler = ShowMessage;
handler += WriteLog;
handler("Hello");
```

Khi một Delegate chứa nhiều method trong invocation list, đó là Multicast Delegate.

2. Action, Func và Predicate

```
Action<string>
→ string -> void

Func<Course, decimal>
→ Course -> decimal

Predicate<Student>
→ Student -> bool
```

- Action: không có return value.
- Func: có return value; generic cuối cùng là kiểu return.
- Predicate<T>: luôn trả bool và thường dùng cho điều kiện/tìm kiếm/lọc.

3. Lambda Expression

Lambda là cú pháp viết anonymous function ngắn gọn. Lambda không phải là Delegate; Lambda có thể được chuyển thành một Delegate tương thích.

```
student => student.Id == "SV01"
```

```

course => course.Credits * course.FeePerCredit

registration =>
{
    Console.WriteLine(registration.Student.FullName);
    Console.WriteLine(registration.Course.Name);
}

```

Hai dạng chính: Expression Lambda và Statement Lambda.

4. Lambda với List<T>

```

Find(Predicate<T>)
→ trả phần tử phù hợp hoặc null

Exists(Predicate<T>)
→ trả bool

FindAll(Predicate<T>)
→ trả List<T>

RemoveAll(Predicate<T>)
→ trả int số phần tử đã xóa

```

Mẹo nhớ: Find dùng khi cần lấy object; Exists dùng khi chỉ cần biết có hay không.

5. Event

Event được xây dựng trên Delegate và giới hạn quyền của code bên ngoài.

```

public event Action<Registration>? CourseRegistered;

registrationService.CourseRegistered += handler;
registrationService.CourseRegistered -= handler;

// Chỉ Publisher raise Event
CourseRegistered?.Invoke(registration);

```

- Bên ngoài được += và -=.
- Bên ngoài không được tùy ý gán bằng =.
- Bên ngoài không được tự Invoke Event.
- Publisher quyết định khi nào Event thực sự xảy ra.

6. Publisher / Subscriber

```

RegistrationService
→ Publisher

CourseRegistered
→ Event

EmailService
LogService
NotificationService
→ Subscribers

SendRegistrationEmail()
WriteRegistrationLog()
ShowRegistrationNotification()
→ Event Handlers

```

Publisher không cần biết cụ thể Subscriber nào tồn tại. Subscriber được kết nối vào Event từ bên ngoài, giúp giảm coupling.

7. Business flow của RegisterCourse()

```

Validate studentId / courseId
    ↓
Trim

```

```

↓
Find Student
↓
Find Course
↓
Check duplicate Registration
↓
Create Registration
↓
Save Registration
↓
Raise CourseRegistered

```

Nguyên tắc quan trọng: Nghiệp vụ chính phải hoàn thành trước khi phát Event thành công. Lưu dữ liệu trước, thông báo sau.

8. Sơ đồ toàn Phần 9

```

Delegate
├── Custom Delegate
├── Multicast Delegate
├── Action
├── Func
├── Predicate
│   └── Lambda
│       └── Collections
└── Event
    └── Publisher / Subscriber

```

9. Kết luận

Sau bài 9.10, các nội dung cốt lõi của Phần 9 đã được thực hành trong cùng một project: Custom Delegate, Multicast Delegate, Action, Func, Predicate<T>, Lambda, Statement Lambda, Event, Publisher/Subscriber, List<T>, validation và exception.

Điểm cần nhớ nhất trước khi sang LINQ: phân biệt Find và Exists, hiểu Lambda tương thích Delegate theo chữ ký, Event dựa trên Delegate, Publisher là bên sở hữu/raise Event, Subscriber chỉ đăng ký handler, và Event thành công chỉ nên được phát sau khi nghiệp vụ chính đã hoàn tất.